



Ανάπτυξη και Ασφάλεια Πληροφοριακών Συστημάτων

Αλυσίδες Καταχωρήσεων & Ευφυείς Συμβάσεις

CryptoSOS

2025-2026

Ελένη Κεχριώτη
f3312503

Όλγα Μαργέλη
f3312504

Το smart contract CryptoSOS υλοποιεί τη λογική του παιχνιδιού SOS σε ένα πίνακα 3x3 όπου μόνο δύο παίκτες μπορούν να παίξουν την φορά. Η λειτουργικότητα του συμβολαίου περιγράφεται παρακάτω:

- **Συμμετοχή Παικτών:** Οι παίκτες εισέρχονται στο παιχνίδι καλώντας τη συνάρτηση join και στέλνοντας ακριβώς **1 Ether**. Το παιχνίδι ξεκινά αυτόματα μόλις συμμετάσχουν δύο παίκτες. Αν προσπαθήσει να παίξει τρίτος παίκτης, απορρίπτεται εφόσον υπάρχει ενεργό παιχνίδι.
- **Τοποθέτηση Συμβόλων:** Οι παίκτες τοποθετούν τα σύμβολα "S" ή "O" εναλλάξ στο ταμπλό δηλώνοντας 1 ή 2 αντίστοιχα. Η επιλογή θέσης γίνεται με δείκτες από το 1 έως το 9. Μετά από κάθε κίνηση, το συμβόλαιο ελέγχει άμεσα αν υπάρχει νικητής (σχηματισμός SOS).
- **Χειρισμός Καθυστερήσης:** Υλοποιήθηκαν μηχανισμοί προστασίας από καθυστερήσεις:

- Αν ένας παίκτης έχει ξεκινήσει το παιχνίδι και περιμένει αντίπαλο για περισσότερο από **2 λεπτά**, μπορεί να καλέσει τη συνάρτηση cancel ώστε να βγει από το παιχνίδι και να έχει πλήρη επιστροφή χρημάτων.
- Αν ένας παίκτης καθυστερήσει πάνω από **1 λεπτό**, ο αντίπαλος μπορεί να καλέσει την tooSlow και να κερδίσει.
- Αν και οι δύο καθυστερήσουν (συνολική αδράνεια) για πάνω από **5 λεπτά**, ο ιδιοκτήτης τερματίζει το παιχνίδι ως ισοπαλία, επιστρέφοντας τα χρήματα (μείον την προμήθεια) στους παίκτες.
- **Ολοκλήρωση Παιχνιδιού:** Ένα παιχνίδι μπορεί να τερματιστεί με έναν από τους παρακάτω τρόπους:
 - Ένας από τους δύο παίκτες να σχηματίσει SOS (**νίκη**).
 - Να γεμίσει το ταμπλό (**ισοπαλία**).
 - Ένας από τους παίκτες να καλέσει επιτυχώς την tooSlow αν ο αντίπαλός του αργεί να παίζει.
 - Ο owner του smart contract καλεί επιτυχώς την tooSlow αν κανένας από τους παίκτες δεν έχει κάνει πρόσφατα κάποια κίνηση.
- **Επαναφορά Παιχνιδιού:** Μετά τη λήξη κάθε παρτίδας, η κατάσταση του παιχνιδιού επαναφέρεται πλήρως, ώστε να είναι δυνατή η έναρξη νέου γύρου.
- **Διαχείριση Κερδών:** Ο ιδιοκτήτης του συμβολαίου μπορεί να αποσύρει τα συσσωρευμένα κέρδη μέσω της συνάρτησης sweepBalance, δηλώνοντας το ποσό σε wei, υπό την προϋπόθεση ότι δεν επηρεάζονται τα δεσμευμένα ποσά ενεργών παιχνιδιών.

Γενικός Σχεδιασμός & Αρχιτεκτονική

Το έξυπνο συμβόλαιο χρησιμοποιεί μια αρχιτεκτονική "Μηχανής Καταστάσεων" (State Machine). Αντί να χρησιμοποιήσουμε μια ρητή μεταβλητή Enum για τις φάσεις του παιχνιδιού, η κατάσταση προκύπτει δυναμικά από τις διευθύνσεις των player1 και player2:

- **Αρχική Κατάσταση:** Όταν ο player1 είναι address(0).
- **Κατάσταση Αναμονής:** Όταν ο player1 έχει οριστεί, αλλά ο player2 είναι address(0).
- **Κατάσταση Παιχνιδιού:** Όταν και οι δύο παίκτες έχουν οριστεί.

Αυτή η σχεδιαστική επιλογή απλοποιεί τη διαχείριση της κατάστασης και μειώνει την ανάγκη για επιπλέον μεταβλητές στο storage.

Βοηθητικές Εσωτερικές Συναρτήσεις

Για καθαρό κώδικα, χρησιμοποιήσαμε τις εξής internal συναρτήσεις:

- **_makeMove**: Ενσωματώνει τη λογική κίνησης, ελέγχους εγκυρότητας και ενημέρωση κατάστασης.
- **_checkSOS**: Ελέγχει (pure function) τις 8 πιθανές γραμμές νίκης.
- **_endGame**: Διαχειρίζεται τον τερματισμό, τα events και τις πληρωμές.
- **_resetGame**: Καθαρίζει όλες τις μεταβλητές για νέα παρτίδα.

Βελτιστοποιήσεις Gas & Χώρου

Η ελαχιστοποίηση του κόστους gas αποτέλεσε προτεραιότητα κατά την ανάπτυξη:

- **Συμπίεση Μεταβλητών**: Χρησιμοποιήσαμε τον τύπο uint64 για τις χρονοσημάνσεις firstJoinTime και lastMoveTime αντί του τυπικού uint256. Αυτό επιτρέπει στη Solidity να "πακετάρει" αυτές τις μεταβλητές στα ίδια slots αποθήκευσης (storage slots) με τις διευθύνσεις (player1, player2) και τη μεταβλητή currentPlayer (uint8). Αυτό μειώνει σημαντικά το κόστος αποθήκευσης (SSTORE/SLOAD) καθώς χρησιμοποιούνται λιγότερα slots των 32-byte.
- **Προσωρινή Αποθήκευση στη Μνήμη (Memory Caching)**: Στη συνάρτηση _makeMove, ο πίνακας κατάστασης board αντιγράφεται στη μνήμη (uint8[9] memory mBoard = board;) πριν περάσει στη συνάρτηση _checkSOS. Δεδομένου ότι η _checkSOS προσπελαύνει τον πίνακα πολλές φορές για να ελέγξει όλες τις γραμμές, στήλες και διαγωνίους, η ανάγνωση από τη μνήμη είναι πολύ πιο οικονομική σε gas από την ανάγνωση από το storage.
- **Αποδοτικός Έλεγχος SOS**: Η συνάρτηση _checkSOS χρησιμοποιεί "ξεδιπλωμένους" ελέγχους αντί για βρόχους επανάληψης. Επιπλέον, δίνει προτεραιότητα στον έλεγχο του

κεντρικού τετραγώνου (θέση 4) για το σύμβολο "O", επιτρέποντας γρηγορότερη εκτέλεση.

Θέματα Ασφαλείας

1. Ασφαλής Διαχείριση Πληρωμών

Η βασική δυσκολία ήταν η σωστή και ασφαλής διαχείριση των πληρωμών (νίκη, ισοπαλία, ακύρωση). Το contract χρησιμοποιεί αυτόματες πληρωμές, δηλαδή τα ether αποστέλλονται άμεσα στους παίκτες μέσα από τις συναρτήσεις που τερματίζουν το παιχνίδι. Αυτό απαιτούσε προσοχή ώστε να αποφευχθούν επιθέσεις επανεισόδου και ασυνέπεια κατάστασης. Επιπλέον, εξασφαλίζουμε ότι ανά πάσα στιγμή το contract έχει αρκετό υπόλοιπο ώστε να αποζημιώσει τους παίκτες σε κάθε περίπτωση.

2. Αντιμετώπιση Reentrancy

Για την αποτροπή επιθέσεων reentrancy, βεβαιωνόμαστε ότι όλες οι μεταβλητές κατάστασης ενημερώνονται ή μηδενίζονται πριν από κάθε εξωτερική κλήση call και το παιχνίδι επαναφέρεται στην αρχική κατάσταση πριν γίνει οποιαδήποτε πληρωμή. Με αυτό τον τρόπο αποτρέπεται η δυνατότητα επαναλαμβανόμενων πληρωμών.

3. Χρονικοί Περιορισμοί

Ο owner και οι παίκτες μπορούν να τερματίσουν παιχνίδια που δεν είναι ενεργά, ώστε να μην μένει για πάντα το contract σε μια κατάσταση. Η μεταβλητή block.timestamp χρησιμοποιείται αποκλειστικά για timeouts. Δεν χρησιμοποιείται για την λήψη οικονομικών αποφάσεων ούτε για δημιουργία τυχαιότητας.

4. Άρνηση Υπηρεσίας (DoS)

Με την μέθοδο tooSlow() εξασφαλίζουμε ότι το παιχνίδι δεν θα μπορεί να “κλειδωθεί” από αδρανείς παίκτες για αόριστο χρόνο, επιτρέποντας το παιχνίδι να συνεχιστεί είτε με επιβολή νίκης είτε με επιβολή ισοπαλίας σε συγκεκριμένα χρονικά πλαίσια.

Παρατηρήσεις

1. Αναγνωρίζουμε ότι η άμεση αποστολή Ether στο τέλος του παιχνιδιού ενέχει έναν κίνδυνο DoS. Σε περίπτωση που ο παραλήπτης είναι ένα κακόβουλο smart contract και απορρίψει την πληρωμή κατά το receive. Έτσι, η συναλλαγή τερματισμού, που απαιτεί require(success), θα αποτύχει και το παιχνίδι δεν θα μπορούσε να ολοκληρωθεί και να επανέλθει στην αρχική του κατάσταση (locked). Σε ένα περιβάλλον παραγωγής θα προσθέταμε μια μέθοδο withdraw, την οποία θα καλούσε κάθε παίκτης για να παραλάβει

τα κέρδη του. Ο κίνδυνος αυτός, αναγνωρίζεται ως περιορισμός των αυτόματων πληρωμών και αποτελεί συνειδητό σχεδιαστικό συμβιβασμό, όπως συζητήθηκε στο μάθημα.

2. Τα events χρησιμοποιούνται αποκλειστικά για καταγραφή (logging) και διαφάνεια και μπορούν να βρεθούν στα logs ενός transaction.